

Výstupy ŠVP

INF-09 Ověřím správnost, najdu a opravím případnou chybu v algoritmu, otestuji, odladím a optimalizuji program. (Informatika 4, 2/2)

Učivo: testování, optimalizace – syntaktické, běhové a logické (funkční) chyby, krokování a ladění programu

Základní pojmy

Test – Dvojice vstup + očekávaný výstup. Očekávaný výstup určíme předem (ručně, zkouškou), pak ho porovnáme se skutečným.

Testovací sada – Soubor testů, který pokryje každou větev programu, hraniční případy i nesmyslné vstupy.

Hraniční případ – Vstup na hranici mezi větvemi nebo na okraji rozsahu: $D = 0$, $a = 0$, věk přesně 15, nula, záporné a velmi velké číslo, prázdný text.

Syntaktická chyba – Porušení pravidel zápisu jazyka (chybí závorka, dvojtečka, odsazení). Program se vůbec nespustí: `SyntaxError`, `IndentationError`.

Běhová chyba – Nastane až za běhu pro určitá data a program spadne: `ZeroDivisionError`, `ValueError`, `NameError`, `TypeError`.

Logická (funkční) chyba – Program doběhne, ale výsledek je špatně. Odhalí ji jen porovnání s očekávaným výstupem.

Traceback – Výpis chyby: soubor, číslo řádku, řádek kódu a na konci typ a popis chyby.

Ladění (debugging) – Hledání příčiny chyby a její oprava; pomáhá kontrolní print, krokování a trasovací tabulka.

Krokování – Provádění programu příkaz po příkazu se sledováním hodnot proměnných (Thonny: Debug, krok přes / krok do).

assert – `assert` podmínka, "zpráva" – automatický test; když podmínka neplatí, program skončí chybou `AssertionError`.

Optimalizace – Úprava programu, aby byl rychlejší, úspornější nebo čitelnější – při zachování stejných výsledků (testy musí dál procházet).

Tři typy chyb

Typ	Kdy se projeví	Příklad	Jak ji najdu
syntaktická	před spuštěním	<code>print("Ahoj"</code> (chybí závorka)	hláška <code>SyntaxError</code> s číslem řádku (zkontroluj i řádek nad ním)
běhová	za běhu, jen pro některá data	<code>x = -c / b</code> pro <code>b = 0</code>	traceback; testy hraničních a nesmyslných vstupů
logická	program nespadne, výsledek neseď	<code>(a + b + c / 3)</code>	porovnání s očekávaným výsledkem, krokování, kontrolní print

Poučky a postupy

- Očekávaný výsledek si urči předem (ručně, zkouškou jako v matematice), teprve pak spustí program.
- Sada testů: aspoň jeden test na každou větev + hraniční případy + nesmyslné vstupy (text, desetinná čárka).
- Hlášku čti odspodu: poslední řádek = typ a popis chyby, nad ním místo; skutečná příčina může být i o řádek výš.
- Logickou chybu hledej zužováním: kontrolní print proměnných, krokování, trasovací tabulka, ověření částí zvlášť.
- Po každé opravě spustí znovu celou sadu – oprava jedné chyby může vyvolat jinou.
- Testy ukážou přítomnost chyby, ne její nepřítomnost: ani sto úspěšných testů nedokazuje správnost.
- Nečíslný vstup ošetří: `try: ... except ValueError: ...` – program nespadne a vypíše srozumitelné upozornění.

Pozor na

- „U mě to funguje“ – jeden úspěšný test nestačí; testuj hranice ($D = 0$, $a = 0$, věk 15).
- `IndentationError`: chybějící odsazení za dvojtečkou nebo míchání tabulátorů a mezer.
- Neopravuj symptom (`if b == 0: b = 1`), ale příčinu – chybějící větev programu.
- Kontrolní print po odladění smaž nebo zakomentuj.
- Text chybových hlášek se mezi verzemi Pythonu liší – typ chyby (`SyntaxError`, `ValueError`...) zůstává.

Vzorový příklad

Program má vypsát průměr tří známek: `prumer = a + b + c / 3`. Pro známky 1, 2, 3 vypíše 4.0. Najdi a oprav chybu.

- Očekávaný výsledek ručně: $\frac{1+2+3}{3} = 2$; program vypsál 4.0 a nespádl → logická chyba.
- Krokování (nebo `print(c / 3)`) ukáže, že se nejdřív spočítá `c / 3 = 1.0` a pak `1 + 2 + 1 = 4`.
- Příčina: dělení má přednost před sčítáním. Oprava: `prumer = (a + b + c) / 3`.
- Znovu celá sada: (1, 2, 3) → 2.0; (1, 1, 1) → 1.0; (5, 5, 4) → 4.666...; všechny testy prošly.